SCRAP AND STEEL

Full-Stack Development Roadmap

Physics-based sandbox / 1v1 arena fighter

Browser-first architecture for games.andrenijman.com

Pillar	Decision
Frontend	TypeScript + Vite + Three.js. Physics engine bake-off, with Rapier 3D as the default candidate.
Multiplayer	Plain WebSocket + JSON. Cloudflare Worker + Lobby Registry DO + per-room DO, matching the established relay pattern.
Hosting	GitHub Pages for static assets; Cloudflare custom relay domain for real-time multiplayer.
Implementation agent	GLM-5.3-Flash as the primary coding agent, constrained by small vertical slices, automated tests, and commit-level review gates.
Design rule	Realistic consequences, simplified interfaces. Engineering choices should create strategy rather than chores.

Advanced weapons are game abstractions

Advanced ordnance uses fictional payload, guidance, propulsion, trigger and mounting modules. Simulate mass, blast force, heat, recoil, arming delay, reliability and resource trade-offs, but do not encode real explosive chemistry or real-world weapon-construction procedures.

Prepared 3 September 2026

Implementation roadmap, architecture specification, balance guardrails, test plan, and deployment plan.

SECTION 0

How to use this roadmap

Work top-to-bottom. Each later milestone assumes the invariants and test gates established earlier.

Section	Deliverable
1	North star and rechecked design decisions
2	Game loop and lobby settings
3	Build Room UX and editor rules
4	Complete modular parts library
5	Power, wiring, control and sensors
6	Locomotion, transmission, fluids and cooling
7	Weapons and Advanced Ordnance Workshop
8	Test Bay and blueprint snapshot system
9	Physics, stress, heat and damage model
10	Arena combat, defeat logic and hazards
11	Multiplayer and Cloudflare architecture
12	Protocol, synchronization, reconnect and desync
13	Client codebase and data architecture
14	Performance budgets and browser support
15	Testing strategy and quality gates
16	GitHub Pages, Cloudflare deployment and secrets
17	GLM-5.3-Flash development workflow
18	Milestone roadmap and Definition of Done
19	Risk register and first implementation tickets
20	Technical references

Most important sequencing rule

Do not begin by building the full parts catalog. First prove one complete vertical slice: frame + battery + controller + motor + wheel + wiring + keybind + test reset + damage + one online fight. After that, expand content through data.

SECTION 1

North star and rechecked design decisions

The game should feel like fast engineering under pressure, not a CAD certification and not an armor-box contest.

Player promise

A player should be able to invent a machine, wire it, bind controls, test it, discover a weakness, fix it, and then understand exactly why it survives or fails in combat. The strongest builds should emerge from meaningful trade-offs rather than one solved layout.

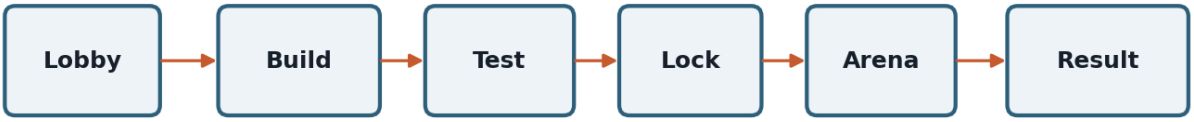
Topic	Locked roadmap decision
Build time	Host types 1-15 minutes; default 7. The room DO stores an absolute deadline so clients cannot pause or drift the clock.
Test Bay	Available during build. Build time continues while testing. End Test instantly restores the exact pre-test blueprint, even if the test robot burned, broke or exploded.
Resource budget	Host selects preset or custom Scrap Points. Suggested starting presets: 700 Light, 1000 Standard, 1400 Heavy. Unlimited is sandbox-only.
Arena hazards	Host toggle, default False. Hazards are a variant, not a substitute for physics balance.
Advanced ordnance	Host toggle, default False. Built in a separate sub-builder and intentionally costly, heavy and control-intensive.
Battery depletion	Power loss alone is not destruction. A powerless robot remains in play and can be attacked. If both become merely energy-dead, the combat timer ends the stalemate as a draw.
Defeat	Requires physical loss of meaningful mobility and meaningful offensive capability, continuously for a confirmation window. Temporary stalls, overheating cooldown or zero battery do not by themselves satisfy defeat.
Combat limit	Default 6 minutes, adjustable 2-10. If nobody is physically defeated at expiry, result is a draw rather than subjective damage judging.
Blueprint visibility	Hidden until both players lock in; optional Open Workshop setting for casual matches.
Realism boundary	Simulate mass, inertia, power, heat, joints, wires, cooling and damage. Abstract details that add setup time but little strategy.

> **Anti-box rule**
> Do not add an invisible anti-box debuff. Thick sealed armor should lose through its actual consequences: mass, higher drive demand, thermal trapping, more expensive structure, joint loads, poor recovery geometry and optional hazards.

SECTION 2

Game loop and lobby settings

Reuse the TUNG lobby experience and protocol conventions, then add engineering-specific match settings.



Testing consumes build time. End Test restores the exact pre-test snapshot.

Online match state machine

- LOBBY - Create or join by room code. Host edits settings. Both players can inspect ping and relay endpoint.
- BUILD - Each player builds locally. Room DO owns the deadline and ready state, not the client.
- LOCKED - Client canonicalizes the blueprint, validates it, sends hash + summary, then final blueprint payload. Changes are disabled after lock.
- COUNTDOWN - Room freezes settings, assigns a match seed and physics authority, distributes both blueprints, and starts a synchronized countdown.
- COMBAT - Clients simulate; room DO forwards input frames, batched correction snapshots, checksums, connection events and match-state transitions.
- RESULT - Relay records result reason and rematch state. Rebuild is the default rematch mode.

Setting	Host control	Default
Build time	1-15 minute numeric field	7 min
Resource budget	Preset + custom 300-2000 SP	1000 SP
Arena	Foundry / Grid / Pit / random	Foundry
Hazards	Floor saws, flame pits, hammer	False
Advanced ordnance	Separate sub-builder and parts	False
Combat limit	2-10 minutes	6 min
Blueprint visibility	Hidden / Open Workshop	Hidden
Part limit	60 / 90 / 120 / custom	120
Spectators	Post-launch feature	False
Rematch	Rebuild / same blueprints	Rebuild

TUNG conventions to preserve

- GET {relay}/lobbies for listing and code-based create/join behavior.
- WebSocket query parameters for create/join room flow, normalized behind a shared client helper.
- ?relay= URL override, window.SCRAP_STEEL_RELAY_URL override, and localhost fallback for local Worker development.
- Host settings broadcast immediately; once both players lock, the relay rejects setting mutations until the round returns to lobby/build.

SECTION 3

Build Room UX and editor rules

The editor must make unconventional robots fast to create while preserving structural and electrical consequences.

Area	Purpose
3D viewport	Orbit/pan/focus camera, placement ghosts, attachment hints, dimension overlay, center-of-mass display, heat/power overlays.
Parts Bin	Search, category tabs, favorites, recent parts. Every major system has its own tab; no mixed catch-all list.
Inspector	Transform, material variant, joint/weld strength, port status, cost, mass, health, temperature and subsystem data.
Tool strip	Select, translate, rotate, weld, joint, wire, hose, duplicate, mirror, delete, undo, redo, Start Test.
Status bar	Time remaining, Scrap Points, mass, estimated idle/peak power, thermal warnings, part count, lock-in state.

Editor rules that protect fun

- Gridless placement with smart snap to faces, axes, shafts, sockets and symmetry planes. Holding a modifier disables snap.
- Floating parts are allowed while editing but are a lock-in error unless attached to the main assembly or intentionally stored inside a valid ordnance subassembly.
- Welds are explicit breakable structural connections. Hinges, sliders, bearings, shafts and couplers are typed joints.
- Collision preview prevents impossible functional overlaps. Decorative meshes can overlap slightly; physics colliders cannot.
- Undo/redo is transactional and includes parts, joints, wires, hoses, control mappings and nested subassemblies.
- Mirror, duplicate, multi-select and alignment tools are mandatory. Repetitive placement is not a skill test worth preserving.
- Blueprint autosave runs locally after every stable edit transaction, never every mouse movement.

Preflight assistant

Severity	Examples
Blocker	No control core; no structurally connected root; blueprint outside spawn volume; impossible collider overlap; corrupted nested subassembly.
Critical warning	No controllable mobility; motor not powered; weapon has no trigger; controller has no source; free-floating battery; broken power bus.
Engineering warning	Peak current above wire/controller capacity; high center of mass; estimated thermal runaway; weak weld margin; severe left/right drive mismatch.
Suggestion	Redundant wire run; unused sensor; excess idle draw; duplicate controller; optional symmetry recommendation.

> **Usability rule**
> Warnings explain what is wrong and highlight the affected graph path. They do not silently auto-wire or auto-design the robot.

SECTION 4

Complete modular parts library

A broad catalog prevents one correct robot shape. Most items are data-driven variants of shared behaviors.

Cost, mass and numerical balance stay in data files and should be telemetry-tuned. The catalog below is the launch target; the first playable vertical slice uses only a small subset.

4.1 Frame and chassis

Part	Gameplay function / trade-off
Mild-steel square tube	Cheap, strong, heavy general frame member.
Chromoly round tube	Higher strength-to-mass; costs more; awkward flat mounting.
Aluminum extrusion	Light and easy to work with; lower impact tolerance.
Titanium angle	Expensive high-strength corner structure.
Carbon-composite spar	Very light and stiff; brittle under sharp impact.
Cross brace	Fast diagonal reinforcement between frame nodes.
Gusset plate	Raises joint load margin at frame intersections.
Bulkhead plate	Creates a strong internal wall and mounting plane.
Electronics tray	Light mounting plate for power/control internals.
Skid rail	Sacrificial underside rail for floor impacts.
Shock cradle	Compliant internal mount that protects fragile electronics.
Roll hoop	Tall protective frame member; adds top weight.

4.2 Armor and external protection

Part	Gameplay function / trade-off
Mild-steel plate	Cheap armor; heavy.
Hardened steel plate	Excellent wear/impact resistance; high cost and mass.
Titanium plate	Strong, lighter, very expensive.
Aluminum plate	Low mass; dents and tears sooner.
Polycarbonate panel	Light transparent cover; useful over electronics, weak to heat.
Composite laminate	Good mass efficiency; brittle when repeatedly struck.
Spaced armor bracket	Creates a sacrificial gap; increases size and joint count.
Sloped wedge plate	Deflects contact and doubles as ground geometry.
Sacrificial strip	Cheap replaceable outer member that absorbs initial hits.
Mesh guard	Keeps debris out while preserving airflow; little impact protection.

SECTION 4.3

Joints, shafts and mechanical attachment

These parts define how loads move through the robot and where failures can occur.

Joints and attachment

Part	Gameplay function / trade-off
Weld seam	Default rigid connection; strength depends on length/material and heat/damage state.
Heavy weld bead	More strength and cost/mass; slower to place only through higher resource cost, not real-time welding.
Hinge joint	Single-axis rotation for arms and flippers.
Pin joint	Compact pivot with lower load capacity.
Bearing block	Supports a rotating shaft and reduces friction.
Linear slider	Constrained translation for lifters and suspension.
Linear rail	Stiffer slider with more mass.
Shaft coupler	Rigidly joins aligned shafts.
Flexible coupling	Tolerates small misalignment at reduced torque capacity.
Universal joint	Transfers rotation through an angle with efficiency loss.
Spring	Stores mechanical energy; useful for suspension and launch mechanisms.
Damper	Dissipates oscillation and reduces bouncing.
Shock mount	Protects internal components from impulse spikes.
Shear pin	Intentional weak link that fails before expensive structure.
Quick-release latch	Detachable mounting point for pods/ordnance; requires a release actuator.
Turntable bearing	Large rotating base for turrets or articulated weapons.

Mounting and geometry helpers

Part	Gameplay function / trade-off
Motor mount	Aligns motor body and shaft with structure.
Battery cage	Adds retention and collision protection around a battery.
Controller bracket	Light electronics mounting interface.
Cable clip	Keeps routed wires away from moving joints.
Hose clamp	Keeps fluid lines attached to frames.
Spacer block	Creates precise clearance without adding a full frame member.
Standoff	Mounts plates/electronics off a surface.
Bumper puck	Compliant contact point that absorbs small impacts.

SECTION 4.4

Power generation, storage and distribution

Power systems should force players to decide between burst performance, endurance, mass and cooling.

Power sources

Part	Gameplay function / trade-off
Compact battery pack	Low mass/capacity; ideal for small robots.
High-discharge battery	Strong burst output; heats quickly under sustained load.
High-capacity battery	Long endurance; heavy.
Armored battery pack	More impact protection with additional mass/cost.
Supercapacitor bank	Excellent short bursts; poor total energy.
Generator module	Steady power from a fuel-consuming engine module; noise, heat and weight.
Fuel-cell module	Efficient steady power; expensive and fragile.
Solar panel	Very low continuous trickle; emergency electronics/slow recharge only.
Regenerative module	Recovers a small amount from braking/spindown when configured with compatible controllers.

Power distribution and wiring

Part	Gameplay function / trade-off
Main power bus	Central high-current distribution node.
Branch bus	Splits power near subsystems; saves long cable runs.
Main disconnect	Logical master power isolation component.
Fuse module	Sacrificial protection against sustained overload.
Resettable breaker	Heavier/costlier protection that can recover after cooldown.
Contactor	Control-switches a high-power branch.
Motor controller	Converts control commands into motor power.
Bidirectional controller	Supports powered forward/reverse with added cost/heat.
DC converter	Feeds low-voltage electronics from the main source.
Light wire	Low mass/cost, low current capacity.
Medium wire	General-purpose power/control wiring.
Heavy cable	High-current drive/weapon path; heavy and harder to route.
Signal wire	Very low mass for sensors/control only.
Shielded signal cable	Resists noise from high-power systems; higher cost.
Connector block	Neat branch point with modest mass.
Slip ring	Carries power/signals across continuously rotating joints.
Cable loom	Bundles multiple wires; slightly more protected and easier to manage.

SECTION 4.5

Control, computing and sensors

Robots are inert until the player intentionally gives them control paths.

Control and logic

Part	Gameplay function / trade-off
Basic control core	Minimum motherboard-like brain; limited I/O.
Expanded control core	More ports and logic slots; heavier/costlier.
I/O expander	Adds ports without replacing the core.
Motor mixer	Maps steering input across multiple drive motors.
Logic gate board	AND/OR/NOT conditions for advanced controls.
Timer board	Delays, pulses or sequences actions.
Latch board	Toggles a command until reset.
Sensor hub	Aggregates sensor inputs and reduces wire clutter.
Telemetry board	Shows live power/heat/sensor state in HUD.
Redundant control link	Backup route if the primary control bus is severed.
Autonomy module	Runs simple player-authored logic rules; not a general scripting computer.
Ordnance controller	Specialized safe game abstraction for advanced subassemblies.

Sensors

Part	Gameplay function / trade-off
Gyro / IMU	Orientation and rotation rate for stabilization or self-right logic.
Wheel encoder	Wheel/shaft speed feedback.
Limit switch	Detects arm or slider end-stop.
Contact sensor	Detects a physical hit or touchdown.
Range sensor	Short/medium distance estimate for automation.
Proximity sensor	Binary near/far trigger with low complexity.
Thermal sensor	Feeds temperature threshold logic.
Current sensor	Detects electrical overload or stall.
Rotation sensor	Tracks spinner/shaft angle or speed.
Pressure sensor	Monitors hydraulic/pneumatic system pressure.
Light sensor	Supports solar/arena-light experiments; low combat value.
Target tracker	High-cost game sensor used only when advanced ordnance is enabled.

SECTION 5

Power, wiring, control and sensors

Real enough to create engineering mistakes, streamlined enough to complete within the build timer.

Electrical simulation model

- Treat the robot as a directed power/control graph. Sources feed buses; buses feed controllers/converters; controllers feed actuators; the control core sends signals to controllable nodes.
- Each power edge has current capacity, resistance and integrity. Loads request power; the solver allocates available source output, applies cable/controller losses and produces heat from overload and inefficiency.
- Open circuits, severed wires, tripped protection and destroyed buses propagate immediately through the graph.
- Do not implement SPICE, AC waveforms, real battery chemistry or component-level electronics. The purpose is gameplay-readable energy flow.

Wiring UX

- Wire tool starts at a compatible port and highlights valid destinations. Drag creates a visible routed cable with auto-routing around simple geometry and manual control points when needed.
- Wire bundles can be selected and rerouted together. Cable clips give optional protection near moving mechanisms.
- Port colors/icons are consistent: power in/out, control, sensor, fluid and mechanical. Never require memorizing tiny connector labels.
- Power overlay animates active current paths; broken/open paths become visually obvious. Heat overlay shows overloaded wiring and controllers.

Control Core screen

Opening the control core presents a linked-device list and a keybind/logic editor inspired by a linked typewriter: devices advertise actions, and the player connects inputs to actions without writing code.

Panel	Behavior
Inputs	Keyboard/mouse/gamepad actions such as W/S, A/D, Space, Q/E, mouse buttons and custom keys.
Devices	Detected powered devices: motors, actuators, weapon controllers, latches, release mechanisms and logic modules.
Binding	Click an input, click a device action, choose momentary/toggle/axis behavior, then test it live in a safe preview.
Logic	Optional conditions such as "if gyro upside-down AND R pressed -> self-right actuator" or timed sequences.
Diagnostics	Shows unpowered devices, severed control paths, conflicting bindings and actions that can never fire.

> **Accessibility without removing engineering**
> Offer quick-bind templates such as "tank drive" or "arcade drive" only after the player has physically installed and wired compatible hardware. Templates map controls; they do not create missing power or structure.

SECTION 6

Locomotion, transmission, fluids and cooling

The drive system is a design space, not a single wheel motor part.

Motors and actuators

Part	Gameplay function / trade-off
Compact DC motor	Light, low torque and power draw.
Torque motor	Slow, high torque; good for heavy drive or crushers.
High-speed brushless motor	Fast and power-dense; needs controller and cooling.
Geared motor	Integrated reduction; heavier but easy drivetrain.
Servo actuator	Position-controlled rotary action for light mechanisms.
Linear electric actuator	Slow precise push/pull with self-locking behavior.
Hydraulic rotary actuator	High torque from hydraulic circuit.
Hydraulic cylinder	Very high linear force; pump/valve/hoses required.
Pneumatic cylinder	Fast burst motion; limited stored air/pressure budget.
Solenoid latch	Fast small motion for releases and triggers.

Locomotion

Part	Gameplay function / trade-off
Rubber wheel	High grip, simple, general purpose.
Hard wheel	Lower grip, low rolling loss, resilient.
Pneumatic tire	Shock absorption and traction; vulnerable to damage.
Omni wheel	Low sideways resistance; good for holonomic designs.
Mecanum wheel	Full holonomic drive with efficiency/traction penalty.
Tank tread	High contact area and traction; heavy, draggy and mechanically complex.
Sprocket + track link	Custom tracked geometry.
Caster	Passive support wheel with minimal control.
Ball transfer	Very low lateral resistance; weak on hazards.
Powered roller	Compact conveyor-style mobility experiment.
Leg hip joint	Powered joint for walking mechanisms.
Leg knee joint	Second-axis leg articulation.
Foot pad	High-friction ground contact for walkers.
Self-righting arm	Dedicated recovery mechanism; consumes mass and space.

SECTION 6.1

Transmission, fluid power and cooling

Mechanical and thermal supporting parts let players trade complexity for performance.

Transmission

Part	Gameplay function / trade-off
Straight shaft	Transfers torque between components.
Hollow shaft	Lower mass, lower impact strength.
Spur gear pair	Compact ratio change; exposed teeth can be damaged.
Bevel gear pair	Changes shaft direction.
Planetary gearbox	High ratio in compact space; expensive and heat-dense.
Chain drive	Tolerates distance/misalignment; exposed and snag-prone.
Toothed belt drive	Light and quiet; can skip/tear under overload.
Pulley set	General belt routing and ratio change.
Clutch	Disconnects a drivetrain or protects overload.
Torque limiter	Slips above a threshold to protect shafts/joints.
Differential	Allows wheel speed difference; useful but not always desirable in combat.
Flywheel	Stores rotational energy for drive smoothing or mechanisms.
Brake	Dissipates rotational energy into heat.
One-way clutch	Allows freewheel in one direction.
Rack and pinion	Converts rotation into linear motion.

Pneumatics and hydraulics

Part	Gameplay function / trade-off
Hydraulic pump	Creates pressure from electrical/mechanical input.
Hydraulic reservoir	Fluid supply and thermal mass.
Accumulator	Stores short bursts of hydraulic energy.
Valve block	Routes pressure to cylinders/actuators.
Hydraulic hose	Carries fluid; vulnerable when exposed.
Pneumatic compressor	Slowly charges air storage.
Air reservoir	Stores pneumatic energy; heavier at larger capacity.
Pneumatic valve	Fast switching for cylinders.
Air hose	Light fluid connection with lower durability.
Pressure regulator	Caps downstream pressure for control/reliability.

Cooling and fire control

Part	Gameplay function / trade-off
Passive heatsink	Adds surface area; no power draw.
Cooling fan	Forced airflow; vulnerable and power-consuming.
Air duct	Guides airflow through sealed-ish armor layouts.
Radiator	Rejects heat from a liquid loop.
Coolant pump	Moves coolant; adds power draw and failure point.
Cold plate	Conducts heat from a hot motor/controller into coolant.
Heat spreader	Moves heat into surrounding structure.
Vent grille	Improves airflow with armor compromise.
Thermal shield	Reduces heat transfer into a protected component.
Fire suppression canister	Single-use game system that slows internal fire at cost/mass.

SECTION 7

Weapons and Advanced Ordnance Workshop

Weapons must demand support systems. A weapon that is easy to mount should not also be light, efficient, cool and indestructible.

Direct-contact weapons

Part	Gameplay function / trade-off
Wedge / plow	Uses geometry and drive traction rather than a powered mechanism.
Lifting fork	Low-profile leverage; needs actuator or linkage.
Pneumatic flipper	Fast burst; limited by stored pressure and reset time.
Hydraulic lifter	Slower but powerful and controllable.
Vertical disc spinner	High kinetic hit energy; gyroscopic handling and spin-up draw.
Horizontal bar spinner	Large reach and recoil loads; dangerous to own frame.
Drum spinner	Compact contact patch and strong bite.
Eggbeater spinner	Aggressive multi-edge contact; high structural stress.
Circular saw	Sustained cutting damage; lower single-hit impulse.
Hydraulic crusher	Very high force if it can capture an opponent.
Powered hammer	Repeated impact with recoil into chassis.
Rotary mace	Off-axis rotating mass; difficult to balance.
Ramming spike	Cheap passive point-contact weapon; depends on mobility.
Flame projector	Short-range heat weapon using a fuel system; high self-heat and storage risk in game terms.
Electromagnetic grabber	Power-hungry temporary hold against compatible metal surfaces.
Clamp jaw	Mechanical restraint for control-oriented designs.

Advanced Ordnance Workshop

When enabled, selecting a Blueprint item opens a small isolated sub-builder. The player assembles a legal ordnance subassembly from game-only modules, validates it, presses Done, and receives a single placeable subassembly in the main build. The nested blueprint remains editable by reopening it.

Game-only module	Purpose / trade-off
Ordnance shell	Structural body and attachment frame.
Abstract blast core	Determines in-game blast impulse/damage; more cores add mass, cost and handling risk.
Projectile body	Carries a launched payload; different mass/aero profiles.
Thruster module	Provides in-game propulsion and heat; requires power/fuel abstraction.
Guidance board	Consumes cost/space/power to steer a compatible propelled body.
Control fins	Add steering authority and drag.
Target tracker	Feeds the guidance board; expensive and vulnerable.
Timer trigger	Arms/fires after an in-game delay.
Impact trigger	Activates on collision after arming.
Proximity trigger	Activates within configured game distance; advanced setting only.
Remote trigger receiver	Allows a bound control input to activate the device.
Safety interlock	Prevents activation until detached or a condition is met.
Release latch	Detaches the ordnance from the robot.
Launch rail	Constrains initial travel and absorbs some recoil.
Projectile launcher core	Abstract gun-like launcher that converts stored energy into projectile velocity.
Magazine / hopper	Stores game projectiles at a mass and reload-complexity cost.
Feed actuator	Moves projectiles from storage to launcher.
Recoil mount	Protects the chassis by allowing controlled recoil travel.
Stabilized mount	Motorized pointing system with added mass/power/control complexity.

Balance guardrail

Advanced ordnance is off by default because long-range damage can bypass the core fun of robot-to-robot contact engineering. Ship it as a host-selected variant after the base combat loop is already strong.

SECTION 8

Test Bay and blueprint snapshot system

Testing should encourage experimentation without creating repair chores or an unlimited-time exploit.

Exact Test Bay behavior

- Click Start Test -> canonicalize the current blueprint -> save a local immutable snapshot -> instantiate a fresh physics world from that snapshot.
- Build timer keeps running. The build UI becomes read-only while the test world is active.
- Test arena contains flat ground, a ramp, a low step, a shove bumper, a suspended impact block and optional target dummy. No opponent blueprint is visible.

- Player can use their actual control bindings. Live overlays show power, heat, current limiting, weld damage and center-of-mass motion.
- If the test robot becomes destroyed, testing does not auto-exit. Show the reason and keep End Test available.
- Click End Test at any time -> destroy the test physics world -> reload the saved pre-test blueprint -> resume editing with zero repair cost and identical resources.
- At build deadline, an active test ends automatically and the pre-test blueprint is what locks in. Testing can never accidentally submit a damaged test state.

Snapshot invariants

Invariant	Test
Canonical ordering	Two equivalent blueprints serialize to the same canonical JSON order and hash.
No runtime leakage	Damage, temperature, charge, velocities and temporary joint deformation never write back into blueprint data.
Nested safety	Advanced ordnance subassemblies restore exactly, including internal wiring and control mappings.
Undo isolation	Entering/exiting test creates no undo entries; only actual build edits do.
Deterministic spawn	Same blueprint + seed creates the same initial rigid bodies, joints and resource state.

> **Do not implement test reset by "repairing" every part**
> That approach will eventually miss a runtime field. Treat the blueprint as immutable source data and recreate the simulation from it.

SECTION 9

Physics, stress, heat and damage model

Use believable systems with controlled complexity. Every simulated quantity needs a gameplay reason and a debug overlay.

Physics engine decision

Run a short engine bake-off before committing. Start with Rapier 3D because it is WASM-based, exposes rigid bodies/joints and supports continuous collision detection for fast bodies. Compare against Havok only if constraint stability or breakable-joint instrumentation is materially better for the target part counts. The rest of the architecture should depend on an internal PhysicsAdapter so the decision is reversible early.

Fixed simulation contract

- Physics step: fixed 60 Hz. Rendering may be 60/120+ Hz using interpolation.
- Enable CCD selectively on high-speed weapon bodies and projectiles, not every rigid body.
- Use real mass and inertia tensors derived from each part definition and transform. Never use a single arbitrary "weight score" in the solver.
- Cap online standard builds by part count and spawn volume, not by an arbitrary total-mass limit. Resource cost and physical consequences already limit extreme mass.
- All gameplay simulation uses seeded random sources; no Math.random() inside authoritative match logic.

Structural stress model

Do not attempt full finite-element analysis. Model a graph of rigid parts and joints/welds with rated load envelopes. Collision impulses, motor/weapon reaction torque, gravity loads and joint error accumulate fatigue/damage. When a connection exceeds its remaining capacity, it degrades and eventually breaks.

System	Simplified but meaningful model
Welds/joints	Strength by joint type, material pair, connection size and accumulated damage. Visual cracks before failure when practical.
Axles/shafts	Torque + bending overload from collisions and stalled mechanisms; bent state increases friction before failure.
Armor	Per-part health + material response. Impacts transfer impulse and damage to nearby joints; penetrative/cutting tools use contact-time rules.
Internal shock	High acceleration spikes can damage poorly mounted batteries/controllers; shock mounts attenuate the spike.
Detached chunks	Remain physical and can interfere with the arena, but may switch to cheaper collision/rendering mode once asleep.

SECTION 9.1

Thermal, electrical and fire simulation

The sealed-box counter must emerge naturally from heat generation and limited rejection.

Quantity	Implementation
Heat generation	Motor/controller inefficiency, current limiting, braking, weapon friction, engines and fires add heat.
Heat storage	Each component has thermal capacity and temperature.
Conduction	Simplified transfer through physical attachment graph; high-conductivity mounts spread heat better.
Air cooling	Exposed surface + airflow + fans/ducts reduce temperature. Internal enclosed volumes receive reduced airflow.
Liquid cooling	Cold plates -> coolant loop -> pump -> radiator graph; broken hose disables/weakens the loop.
Overheat effects	Performance derating first, then controller shutdown, battery damage or ignition at extreme temperatures.
Fire	Local heat/damage source that can spread through vulnerable adjacent components; suppression reduces duration/intensity.
Electrical overload	Current above rated edge/controller capacity causes voltage drop, heat and protection trips; severe sustained overload damages components.

Center of gravity and drivability

- Compute total center of mass from physical bodies and show it as a live gizmo in the editor.
- Tall/heavy-forward robots should tip because acceleration, wheel contact and inertia create real torque - not because of a scripted tip chance.
- Traction is limited by normal force and wheel/material friction. More motor power beyond traction creates wheelspin and heat rather than free acceleration.
- Gyroscopic spinner effects are real rigid-body angular momentum. Provide a warning overlay for extreme gyro coupling, but do not suppress it.

Debug overlays required before balance work

Overlay	Shows
Mass / CoM	Part masses, total mass, center of mass and support polygon.

Overlay	Shows
Power	Source output, current paths, voltage/power limiting and charge.
Heat	Component temperature, heat generation and cooling paths.
Stress	Joint utilization, recent impulse spikes and fatigue damage.
Control	Input -> logic -> device action paths and unreachable actions.
Network	Tick, RTT, snapshot age, correction magnitude, desync hash state.

SECTION 10

Arena combat, defeat logic and hazards

A match ends on physical incapacity, not a hidden health bar and not simply because a battery is empty.

Defeat evaluator

Evaluate potential physical capability from the surviving assembly graph, not just whether the robot is moving at this instant.

Signal	Counts as operational when...
Mobility	At least one surviving mechanically connected drive or recovery mechanism can still create meaningful chassis translation/rotation if supplied by an intact eligible power path; a merely empty battery does not mark the hardware destroyed.
Offense	At least one surviving weapon path can still produce meaningful contact/projectile/thermal attack if its intact support system can function. A pure ramming robot uses mobility as its offense.
Control	An intact control core + signal path can command the relevant mechanism, or a valid autonomous rule can still do so.
Destroyed state	Both meaningful mobility/recovery and offense are physically unavailable because components/joints/control paths are broken, detached, jammed beyond recovery or destroyed. Condition must persist for 3 seconds.
Double KO	If both satisfy destroyed state within the same 3-second resolution window, result is a draw.
Energy stalemate	If neither is destroyed but no remaining energy source can drive any meaningful action for either robot, allow a short grace period and then draw, or simply reach the combat timer.

Arena set

Arena	Identity
Foundry	Baseline rectangular arena, flat floor, strong walls, hazard sockets disabled by default.
Test Grid	Clean tournament-style arena for competitive matches; minimal visual clutter.
Pitworks	More ramps, raised edges and optional hazard zones; casual variant.
Scrapyard	Asymmetric cover/debris layout, only for casual/custom because it affects fairness.

Optional hazards

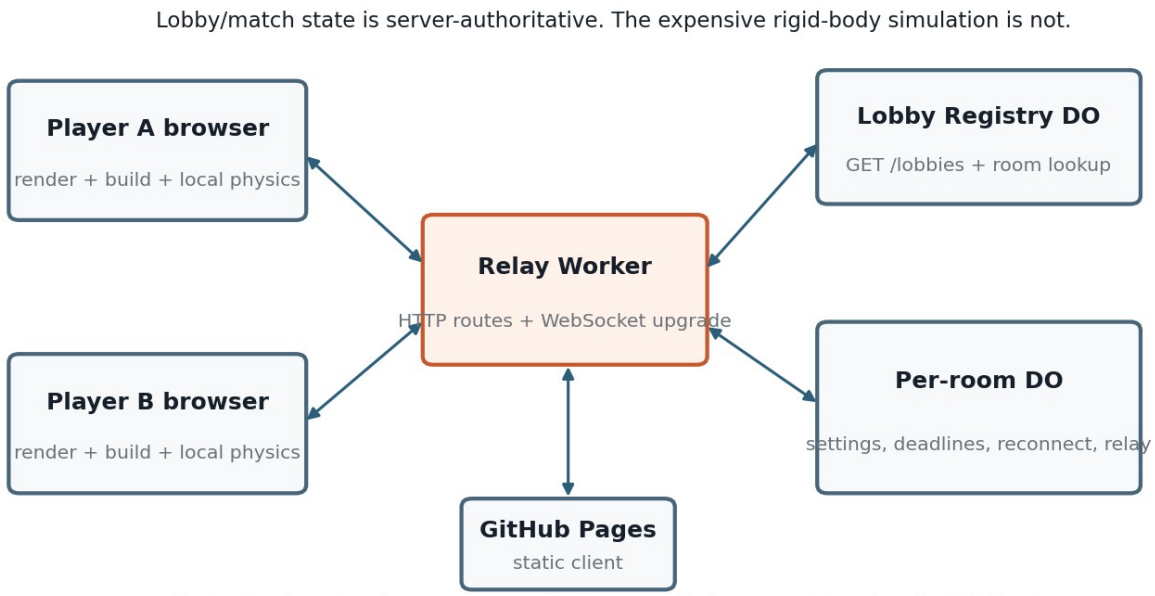
- Kill saws: telegraphed floor slots with strong underside/contact damage.
- Flame pits: timed heat zones that punish immobility and poor cooling.
- Hydraulic hammer: slow, obvious overhead strike with a clear warning shadow/animation.
- Hazards use a match seed and deterministic schedule shared to both clients. They are never secretly randomized per client.

> **Fairness default**
> Hazards default to False. The base game must already defeat lazy armor boxes through mass, power, cooling and structural consequences.

SECTION 11

Multiplayer and Cloudflare architecture

Reuse the proven Worker + Durable Object room pattern, but optimize the hot path for 1v1 physics traffic.



Recommended service layout

Component	Responsibility
Static game	GitHub Pages: HTML, JS chunks, WASM physics build, models, textures, audio and version manifest.
Relay Worker	Origin checks, /lobbies routing, WebSocket upgrades, basic abuse/rate checks, version gate.
Lobby Registry DO	Public room summaries, room-code allocation, cleanup and lookup. No combat simulation.
Per-room DO	Two-player identity, host/settings authority, build deadline, ready/lock state, blueprint handoff, match seed, reconnect tokens, WebSocket relay and result state.
Browsers	Full rigid-body simulation, rendering, build editor and local prediction. One side is designated correction authority per round.

Component	Responsibility
Optional games-guard integration	Not required for first release. Direct custom relay domain avoids double-proxying high-rate snapshots. Add same-origin proxy only if a later security/cookie requirement justifies it.

Domain and endpoint convention

- Production: wss://relay.scrap.andrenijman.com (recommended custom domain).
- Lobby list: GET https://relay.scrap.andrenijman.com/lobbies.
- WebSocket: ?create=1 or ?code=ROOM (internally normalize room/code naming so client code uses one concept).
- Overrides: ?relay=, window.SCRAP_STEEL_RELAY_URL, then localhost fallback in development.
- Origin allowlist: games.andrenijman.com, any exact GitHub Pages preview origin you intentionally support, and localhost development origins.

> **Why the Durable Object should not run 60 Hz physics**
> Cloudflare Durable Object hibernation is valuable for idle rooms, while scheduled timers keep an object active. High-frequency physics also spends CPU on a system that the browser already must run for rendering. Keep the DO authoritative over match state and use it as a validated relay, not as a general game server.

SECTION 12

Protocol, synchronization, reconnect and desync

Treat the wire format as a versioned product surface, not scattered JSON literals.

Protocol envelope

Every message uses a small common envelope: protocol version, type, room, sequence, match/tick when relevant, and payload. Parse through a shared schema validator. Unknown message types are ignored or rejected safely; malformed payloads never reach simulation code.

Direction	Message families
Client -> relay	hello, create_room, join_room, set_settings, set_ready, lock_blueprint, blueprint_chunk, input_frame, checksum, authority_snapshot, rematch, ping
Relay -> client	welcome, lobby_state, settings, build_start, lock_ack, blueprint_manifest, blueprint_chunk, match_countdown, input_frame, snapshot, correction_request, peer_disconnected, peer_reconnected, result, error, pong

Combat synchronization plan

- Local physics runs fixed 60 Hz. Input changes are timestamped/ticked and sent immediately; the relay batches high-frequency forwarding where practical around 50-100 ms without delaying critical transitions.
- Both clients instantiate the same immutable blueprint and match seed. The non-authority client simulates normally for responsiveness rather than rendering only remote snapshots.
- Correction authority sends compact state snapshots at a tunable rate such as 10-15 Hz, chunked when necessary. Include only transforms/velocities and mutable subsystem state required to reconcile.
- Clients send periodic simulation checksums. Small divergence is corrected smoothly; large divergence triggers a hard correction and a visible network warning in debug mode.
- Input/state sequence numbers prevent old messages from being applied after reconnect. All match messages carry a match instance ID so stale packets from a previous round are harmless.
- Authority is selected by the room at match start. Do not assume the lobby host must always be physics authority; pick using deterministic room policy and measured connection quality if desired.

Reconnect behavior

Situation	Rule
Short disconnect during build	Keep slot reserved for 30 seconds; build deadline continues. Rejoining client restores local autosave and room state.
Short disconnect during combat	Pause input acceptance for the missing peer but keep the room alive; allow a 10-15 second reconnect grace. Physics can freeze or enter a clearly signaled pause agreed by both clients.
Authority disconnect	If reconnect occurs within grace, restore from last accepted snapshot. If not, either award disconnect win or draw according to lobby mode; do not attempt risky mid-fight authority migration in v1.
Page refresh	Reconnect token in sessionStorage identifies the player slot; blueprint autosave restores local build data.
Version mismatch	Reject join with a clear "game updated - refresh" response before blueprints are exchanged.

SECTION 13

Client codebase and data architecture

Separate immutable blueprint data from mutable simulation state so test reset, networking and persistence stay reliable.

Recommended stack

Layer	Choice
Language/build	TypeScript, Vite, ESM, strict type checking.
Rendering	Three.js direct scene management. Keep render loop outside UI framework re-render cycles.
UI	Lightweight React or Preact shell for menus/editor panels; state store such as Zustand only for UI/app state, not physics bodies.
Physics	PhysicsAdapter abstraction; Rapier 3D candidate behind it.
Validation	Shared runtime schemas for relay messages and blueprint files.
Tests	Vitest for unit/property tests; Playwright for browser/editor/network flows; Miniflare/Wrangler test harness for Worker/DO.
Assets	glTF/GLB models, compressed textures, audio sprites where sensible; lazy load nonessential catalog assets.
Persistence	IndexedDB for blueprint library and autosaves; localStorage only for tiny preferences.

Repository layout

Path	Purpose
src/app/	Boot, route/screen state, settings and version checks.
src/editor/	Build tools, selection, transforms, snapping, undo/redo, parts bin, inspector.
src/blueprint/	Canonical schema, serialization, hashing, nested subassemblies, migrations.
src/sim/	Physics adapter, rigid-body assembly, fixed tick, stress, damage, heat, electrical and fluid solvers.

Path	Purpose
src/control/	Control graph, keybind editor, logic blocks, device capability discovery.
src/combat/	Arena, hazards, defeat evaluator, result rules.
src/net/	Relay URL resolution, WebSocket client, protocol schemas, chunking, sync and reconnect.
src/render/	Three.js scene, materials, particles, camera, overlays and LOD.
src/content/	Part definitions, arena definitions, balance tables.
worker/	Relay Worker, LobbyRegistry DO, Room DO, schemas and rate/origin checks.
shared/	Protocol/blueprint types safe for client + Worker.
tests/fixtures/	Golden blueprints and deterministic physics scenarios.
docs/	Architecture, protocol, physics, balance and agent instructions.

Blueprint schema principles

- UUID per part, deterministic parent/connection IDs, explicit transforms, variant/material ID, typed ports, joint definitions, wire/hose edges and control graph.
- No runtime values such as temperature, charge, velocity, health or current damage in blueprint data.
- Nested ordnance subassembly is a versioned child blueprint with attachment interface and allowed exposed control/power ports.
- Every schema version has a migration function and golden fixture. Never silently mutate an old saved blueprint without migration.

SECTION 14

Performance budgets and browser support

Performance is a gameplay feature because physics instability and input latency destroy trust in a building game.

Budget	Launch target
Desktop target	Current Firefox and Chromium on Windows/Linux/macOS. Safari is supported if the physics/render test matrix passes; otherwise show an explicit compatibility notice.
Frame rate	60 fps render target on a mid-range desktop with two 90-120 part robots. Never tie simulation speed to render frame rate.
Physics	60 Hz fixed step; adaptive visual quality before reducing simulation tick.
Rigid bodies/joints	Standard online cap 120 parts per robot; measure worst-case joint-rich designs early.
Network	Inputs tiny/on-change; correction snapshots 10-15 Hz target and delta/chunked. Avoid sending mesh/content data during combat.
Initial load	Shell first; lazy-load heavy part models/textures. Keep core physics/render code cacheable and versioned.
Memory	Destroy old test/combat physics worlds fully. Leak tests must include repeated Test Bay cycles and rematches.
Particles/debris	Visual debris pool with hard cap; detached real physics parts sleep and downgrade rendering once inactive.

Graceful quality tiers

- Ultra/High: shadows, reflections, dense particles, high texture resolution.
- Medium: simpler shadows, fewer particles, lower reflection resolution.
- Low: no expensive dynamic shadows, simplified particles, aggressive model LOD. Physics and damage rules stay identical.
- Auto-select from a short startup benchmark, with manual override. Never use browser user-agent as the primary performance decision.

Physics stress test fixtures

Fixture	Purpose
Box monster	120-part dense armored robot to test contacts and thermal enclosure.
Spinner torture	Two high-speed weapon rotors with CCD colliding at odd angles.
Joint forest	Maximum hinges/sliders/welds and deliberate break cascades.
Debris storm	Both robots fragment into many detached pieces.
Walker	Many articulated leg joints under constant motor torque.
Track build	High contact count with multiple tread/ground contacts.
Heat soak	Sealed high-power robot exercising motors and weapons for full combat duration.

SECTION 15

Testing strategy and quality gates

Every physics feature needs deterministic fixtures and every multiplayer phase needs failure-path tests before content expansion.

Test pyramid

Layer	Must cover
Unit	Blueprint canonicalization, resource budget, graph connectivity, power allocation, heat integration, control mapping, defeat logic, message validation.
Property/fuzz	Random valid/invalid blueprints, graph disconnections, malformed JSON messages, NaN/Infinity rejection, chunk ordering and duplicate packets.
Physics fixtures	Known setups for acceleration, tipping, spinner contact, weld break, overheat, wire overload, fluid failure and battery depletion.
Worker/DO	Lobby create/join/list, deadlines, host-only settings, room cleanup, reconnect tokens, origin allowlist, size/rate limits, stale match IDs.
Browser integration	Place/weld/wire/bind/test/reset, autosave/restore, lock-in, fight, reconnect, rematch.
Network simulation	Artificial 50/100/200 ms RTT, jitter, temporary disconnect, duplicated/out-of-order app messages and authority snapshot loss.
Performance	Worst-case part fixtures, repeated Test Bay loops, rematches, memory growth and long combat heat soak.

Non-negotiable release gates

- End Test restores a canonical blueprint hash identical to Start Test snapshot after 100 automated destructive test cycles.

- No NaN/Infinity reaches transforms, physics velocities, heat state, damage state or network payloads in fuzz tests.
- Same blueprint + same seed + same recorded input trace can replay without catastrophic divergence on the supported browser matrix. Exact bit identity is not required if correction logic remains stable.
- Joining a room with an incompatible protocol/game version fails clearly before match start.
- Neither browser can change frozen lobby settings or submit a blueprint exceeding host budget/part limits.
- No client bundle contains Cloudflare credentials, worker secrets or contents from special.special.
- All required controls are keyboard remappable. Gamepad support may follow, but no gameplay action should be trapped on an unremappable hard-coded key.
- Baseline arena is fun with hazards disabled and advanced ordnance disabled.

> **Balance work comes after instrumentation**
> If a player says a design is overpowered, the team should be able to inspect mass, power, heat, stress and match-event telemetry before changing numbers.

SECTION 16

GitHub Pages, Cloudflare deployment and secrets

Ship the static game and relay independently so gameplay assets can be cached aggressively while the room service stays small.

Deployment topology

Artifact	Deployment
Client	Vite production build -> GitHub Pages -> games.andrenijman.com game path.
Relay	Cloudflare Worker + Durable Object bindings -> production relay custom domain.
Staging client	GitHub Pages preview/branch deployment or a separate path using a staging relay override.
Staging relay	Separate Worker name/DO namespace so tests cannot corrupt production lobbies.
Version manifest	Client build includes git SHA + protocol version. Relay exposes compatible protocol/game range.

Secrets policy

- Add special.special to .gitignore and a global/local pre-commit secret scan. The roadmap never requires the coding agent to print or commit that file.
- For local deployment, load only the required scoped Cloudflare API token into CLOUDFLARE_API_TOKEN for the wrangler process. Do not pass the entire secret file into prompts or logs.
- For CI deployment, store a scoped Cloudflare API token and account identifier in GitHub Actions secrets. A local file cannot securely power unattended GitHub-hosted CI.
- Use Worker secrets for relay-only values. Nothing secret is shipped in GitHub Pages JavaScript.
- Origin allowlists are environment variables/config, not secrets. Keep exact production and localhost origins; reject unexpected origins before WebSocket acceptance.
- Prefer a narrowly scoped API token over a global API key. Rotation should not require code changes.

CI/CD gates

On pull request	On main / release
Typecheck + lint + unit tests + blueprint fixtures + Worker tests + Playwright smoke + production build.	All PR gates, then deploy Pages artifact and staging/prod Worker according to branch policy.
Upload test logs/artifacts on failure; do not deploy from a failed test run.	Run post-deploy health check: /lobbies, WebSocket handshake, create/join test room, protocol/version check.

On pull request	On main / release
Secret scan rejects special.special, tokens and known credential patterns.	Tag release with client SHA + Worker SHA + protocol version; keep rollback instructions.

> **Current environment note**
> The runtime used to produce this roadmap does not have the gh executable installed, so no claim is made that the authenticated GitHub CLI state was inspected. The deployment plan assumes your stated local gh authentication and should begin with a repo-level verification on the development machine.

SECTION 17

GLM-5.3-Flash development workflow

Use the model as an implementation agent with explicit contracts, not as a one-shot generator for the whole game.

Repository instructions to create before feature work

File	Contents
AGENTS.md	Architecture invariants, commands, style, testing rules, forbidden secret access, definition of a complete ticket.
docs/ARCHITECTURE.md	Layer boundaries, source-of-truth rules, blueprint vs runtime state, rendering vs simulation.
docs/PROTOCOL.md	Message schemas, state machine, versioning, sequence/tick semantics, reconnect.
docs/PHYSICS.md	Units, fixed timestep, coordinate system, collider rules, stress/damage/thermal/electrical assumptions.
docs/BALANCE.md	Scrap Point philosophy, standard lobby defaults, part categories and telemetry fields.
docs/TESTING.md	Exact commands, golden fixtures, browser matrix and release gates.
docs/DEPLOYMENT.md	Pages + Worker environments, safe secret handling and rollback.

Agent ticket protocol

- One ticket = one vertical behavior or infrastructure slice with explicit acceptance tests. Avoid prompts like "build the physics system".
- Before editing: ask the agent to inspect only relevant files and restate invariants. Large context is available, but unnecessary repo dumps increase accidental cross-cutting edits.
- Require a short implementation plan, then code, then run the exact local tests. The agent must fix failures before the ticket is considered complete.
- Every ticket ends with: changed files, behavior summary, test commands/results, known limitations, and a commit-ready diff. No hidden "TODO later" for acceptance criteria.
- For physics changes, require a deterministic fixture or replay. For networking changes, require malformed-message and reconnect tests. For UI changes, require a Playwright path.
- Keep balance/content additions separate from engine changes. A 50-part content PR should not also rewrite the electrical solver.
- Never expose special.special to the model unless the deployment command itself must access an environment variable derived from it. Secrets do not belong in context.

Suggested GLM prompt skeleton

> **Ticket prompt template**
> Goal: implement one named behavior. Read: exact files/docs. Invariants: list 3-6 non-negotiables. Acceptance tests: explicit observable cases. Commands: typecheck/test/build. Constraints: no unrelated refactors, no secret access, preserve

> protocol/schema compatibility. Finish by running tests and summarizing the diff.

GLM-5.3-Flash is particularly suitable here because its current model card/documentation describes tool calling, reasoning, multimodal input and a very large context window. Use those capabilities for repository navigation, screenshots/debugging and repeated test-fix loops, but still bound each task tightly.

SECTION 18

Milestone roadmap and Definition of Done

Each milestone has a playable outcome. Do not unlock the next content wave until the prior exit gate is green.

Milestone	Focus	Build	Exit gate
M0	Foundation	Repo structure, AGENTS/docs, Vite/TS/Three shell, physics adapter spike, Worker skeleton, CI.	App boots; test cube sim; Worker local room smoke; all checks green.
M1	Offline build vertical slice	Frame, battery, wire, controller, motor, wheel; select/transform/snap/weld; budget; undo/redo.	Player builds a drivable cart from individual systems without hidden auto-connections.
M2	Control + Test Bay	Control Core UI, keybinds, device graph, immutable snapshot, Start/End Test.	Cart can be bound, tested, destroyed, reset to identical blueprint hash.
M3	Damage + anti-box physics	Joint break, armor damage, CoM, power draw, wire limits, heat/cooling, debug overlays.	Heavy sealed design demonstrably pays real mass/power/thermal costs.
M4	Combat vertical slice	Second robot, one spinner, one lifter, baseline arena, defeat evaluator, result screen.	Local 1v1 is fun and produces correct win/draw conditions.
M5	TUNG-style lobby	Registry DO, room DO, lobbies, create/join, settings, deadline, ready/lock, version gate.	Two browsers complete lobby -> synchronized build deadline -> lock-in.
M6	Online combat sync	Input forwarding, authority snapshots, checksums, correction, reconnect, chunking.	Two remote browsers finish multiple matches under latency simulation without unstable divergence.
M7	Content wave 1	Frame/armor/joints, complete drive/transmission, hydraulics/pneumatics, cooling, sensors, 8-10 contact weapons.	At least five meaningfully different competitive archetypes are viable.
M8	Advanced systems	Nested sub-builder, advanced ordnance host toggle, projectile sync, safety/arming logic.	Advanced mode works without affecting default mode balance or protocol stability.
M9	Polish + performance	PBR art pass, sound, particles, onboarding, overlays, quality tiers, accessibility, load optimization.	60 fps target on standard fixture and no build-flow usability blockers.
M10	Deployment hardening	Custom relay domain, prod/staging configs, secrets, monitoring, rollback, Pages workflow.	Production health checks and rollback tested; no secrets in artifacts.
M11	Release validation	Closed playtests, telemetry-driven tuning, bug burn-down, compatibility matrix, docs.	All Definition of Done gates pass; standard mode is stable and fun without optional hazards/ordnance.

Definition of Done for 1.0

- 1v1 lobby flow works on the production relay with room listing, codes, settings, reconnect and rematch.
- 1-15 minute build timer and custom resource budget are server-enforced.

- Players can build unconventional robots from a broad categorized library, with no hidden automatic drivetrain/control assumptions.
- Power, wiring, controls, mass, center of gravity, heat, joint stress and damage all materially affect outcomes and have readable diagnostics.
- Test Bay is safe, instant-reset and does not pause the build deadline.
- Baseline combat determines destruction correctly; battery depletion alone is never mistaken for destruction; simultaneous destruction draws.
- Hazards default False; advanced ordnance default False; both are stable host-selected variants.
- Production deploy uses GitHub Pages + Cloudflare Worker/Durable Objects with a custom relay domain and safe secret handling.
- All automated quality gates and supported-browser tests pass.

SECTION 19

Risk register and first implementation tickets

The hardest problems are physics stability, editor usability and multiplayer correction - not the number of part models.

Risk	Level	Mitigation
Physics instability with many breakable joints	High	Run M0 bake-off and stress fixtures before committing content. Keep PhysicsAdapter. Cap standard online part count.
Build editor too slow for 7 minutes	High	Ship snap/mirror/duplicate/multi-select early; test full build timings with real players before adding complexity.
Network divergence	High	Fixed tick, same WASM build, seeded randomness, checksums, correction snapshots, latency simulation and visible debug telemetry.
One dominant armor/spinner meta	High	Telemetry + physical costs + varied weapon archetypes. Avoid arbitrary counters; tune resource/thermal/structural data.
Advanced ordnance overwhelms base combat	High	Default off, separate milestone, high support costs, independent balance telemetry.
Electrical system becomes tedious	Medium	Port highlighting, auto-routed cable path, bundles, preflight explanations and control templates after hardware exists.
Browser performance	High	Worst-case fixtures at M0/M3, asset lazy loading, visual LOD, selective CCD, detached-part sleeping.
Secret leakage in AI/CI	High	special.special ignored and never prompted; scoped GitHub Actions secret; secret scan; no credentials in client.
Schema saves become incompatible	Medium	Versioned blueprint schema + migrations + golden fixtures from first save format.
Disconnects cause bad match outcomes	Medium	Explicit reconnect grace and result policy; no complex authority migration in v1.
Content production bottleneck	Medium	Data-driven PartDefinition, shared behaviors, procedural material variants, model checklist and batch validation.
Host cheating / authority trust	Medium	Casual v1 accepts limited trust; checksums/shadow sim detect gross mismatch. Do not claim competitive anti-cheat without a server physics architecture.

First 12 implementation tickets

#	Ticket
01	Bootstrap app + CI + strict TS + Three scene + version manifest.
02	PhysicsAdapter spike: Rapier prototype, fixed 60 Hz loop, body/joint fixture, CCD spinner fixture.
03	Blueprint v1 schema + canonical serialization + hash + migration harness.
04	Parts Bin + place/select/move/rotate/smart-snap + frame parts.
05	Explicit weld graph + delete/duplicate/mirror + transactional undo/redo.
06	Battery + power bus + wire routing + motor controller + motor + wheel power graph.
07	Control Core screen + W/S/A/D bindings + motor mixer + diagnostics.
08	Test Bay immutable snapshot + Start/End Test + destructive reset tests.
09	Mass/CoM/traction + weld overload + basic armor damage + debug overlays.
10	Heat + current limits + fan/vent cooling + sealed-box fixture.
11	Local second robot + baseline arena + one spinner + defeat evaluator.
12	Cloudflare Worker/Registry DO/Room DO + lobbies/create/join/settings/build deadline smoke.

> **Do not skip Ticket 08**
> Test reset is a foundational architecture check. If runtime state can leak back into blueprints, networking, autosave, rematches and replays will all become harder later.

SECTION 20

Technical references

Current external facts used to shape the implementation choices; game-specific relay behavior is based on the architecture supplied for games.andrenijman.com.

Reference	URL	Why it matters
GLM-5.3-Flash model card	https://huggingface.co/zai-org/GLM-5.3-Flash	Current model card describes a 320B-total/18B-active multimodal model and links coding/agentic evaluation information.
Databricks GLM-5.3-Flash listing	https://docs.databricks.com/aws/en/machine-learning/foundation-model-apis/supported-models	Current listing documents text/image input and function-calling/reasoning support for the hosted model.
Cloudflare Workers AI GLM-5.3-Flash	https://developers.cloudflare.com/workers-ai/models/	Cloudflare currently lists GLM-5.3-Flash as a hosted Workers AI model; this roadmap does not require using Workers AI for the game itself.
Cloudflare Durable Objects WebSockets	https://developers.cloudflare.com/durable-objects/best-practices/websockets/	Durable Objects support WebSocket servers and hibernation; Cloudflare recommends hibernatable WebSockets for idle cost efficiency and discusses batching high-frequency data.

Reference	URL	Why it matters
Durable Object lifecycle	https://developers.cloudflare.com/durable-objects/concepts/durable-object-lifecycle/	Scheduled setTimeout/setInterval callbacks prevent hibernation, supporting the decision not to run a permanent high-rate physics tick in the room DO.
Durable Object limits	https://developers.cloudflare.com/durable-objects/platform/limits/	Current platform limits and per-invocation CPU behavior.
Rapier JavaScript CCD	https://rapier.rs/docs/user_guides/javascript/rigid_body_ccd/	Rapier documents continuous collision detection for fast rigid bodies, relevant to spinner/projectile prototypes.
Three.js WebGL/WebGPU docs	https://threejs.org/manual/	Current Three.js documentation for browser rendering and compatibility planning.

Final architecture statement

Scrap and Steel should be built as a deterministic, data-driven browser simulation with immutable blueprints, explicit subsystem graphs, fixed-step physics, and a thin Cloudflare room authority. The lobby and relay conventions remain familiar to the existing games site, while the heavy simulation stays where it is most practical: in the game clients. The roadmap intentionally delays advanced ordnance and content breadth until the core engineering loop is already fun, testable and stable online.

> **Success condition**
> A new player can make a simple working robot without a tutorial video; an expert can spend many matches discovering unusual mechanisms that the system genuinely supports.